INFORME DE CALIDAD DE PRODUCTO – SPRINT 2

1. Metadatos

Grupo: Grupo 2
Sprint: Sprint 2
Fecha de realización: 02/11/2026
Hash del Commit (rama develop): 539d1ae

2. Capturas de SonarCloud y ScrumDesk

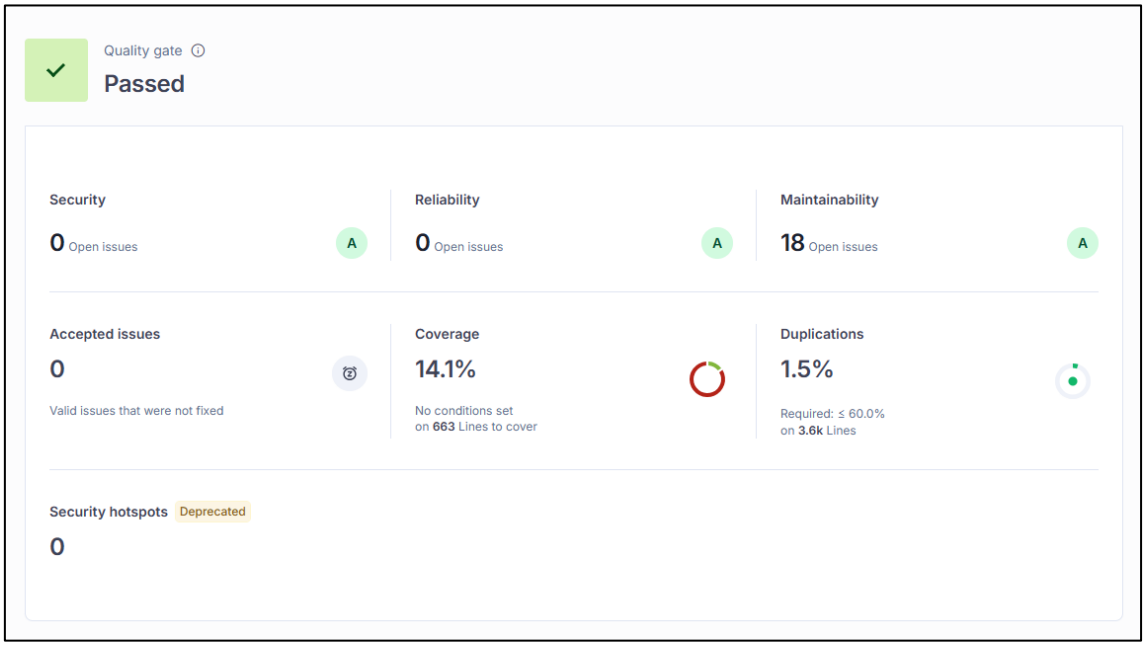


Figura 1: Captura resumen del análisis

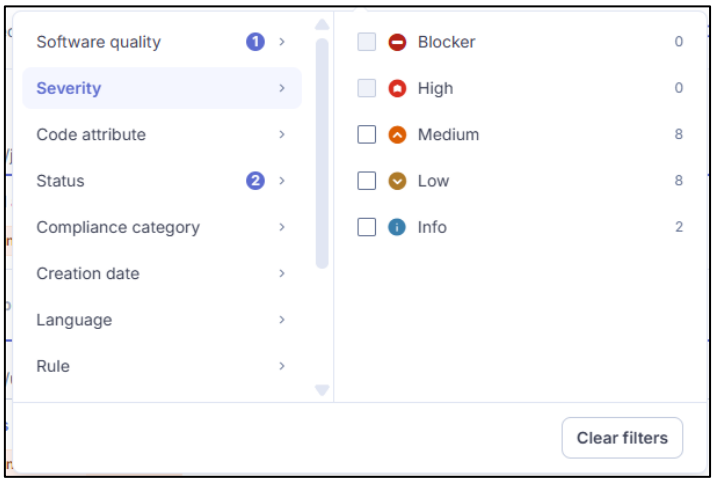


Figura 2: Issue Maintainability clasificado por severidad

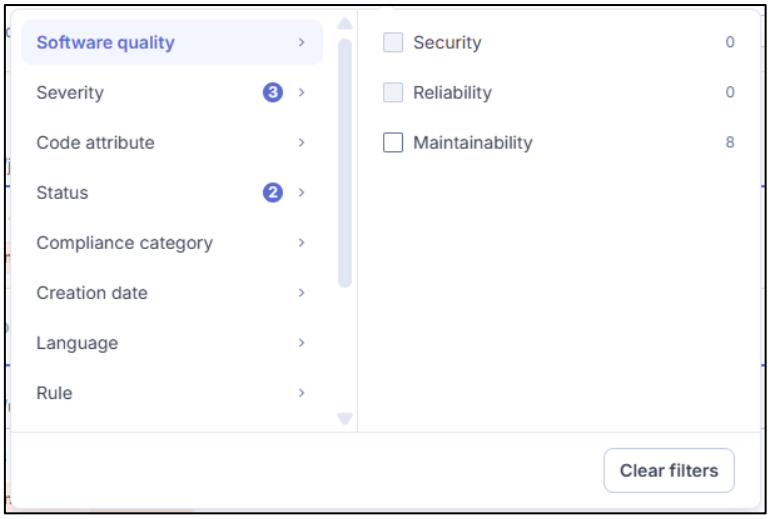


Figura 3: Ausencia de Issues de Security y Reliability

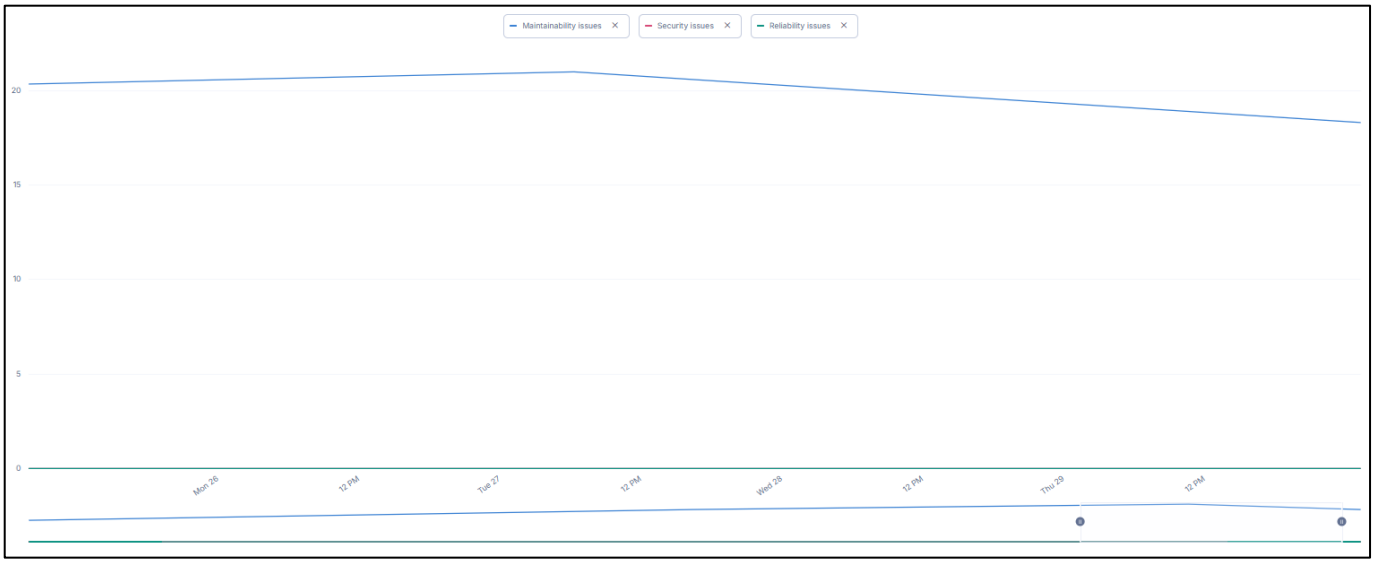


Figura 4:Número de Issues de cada Elemento (Security, Reliability, Maintainability)

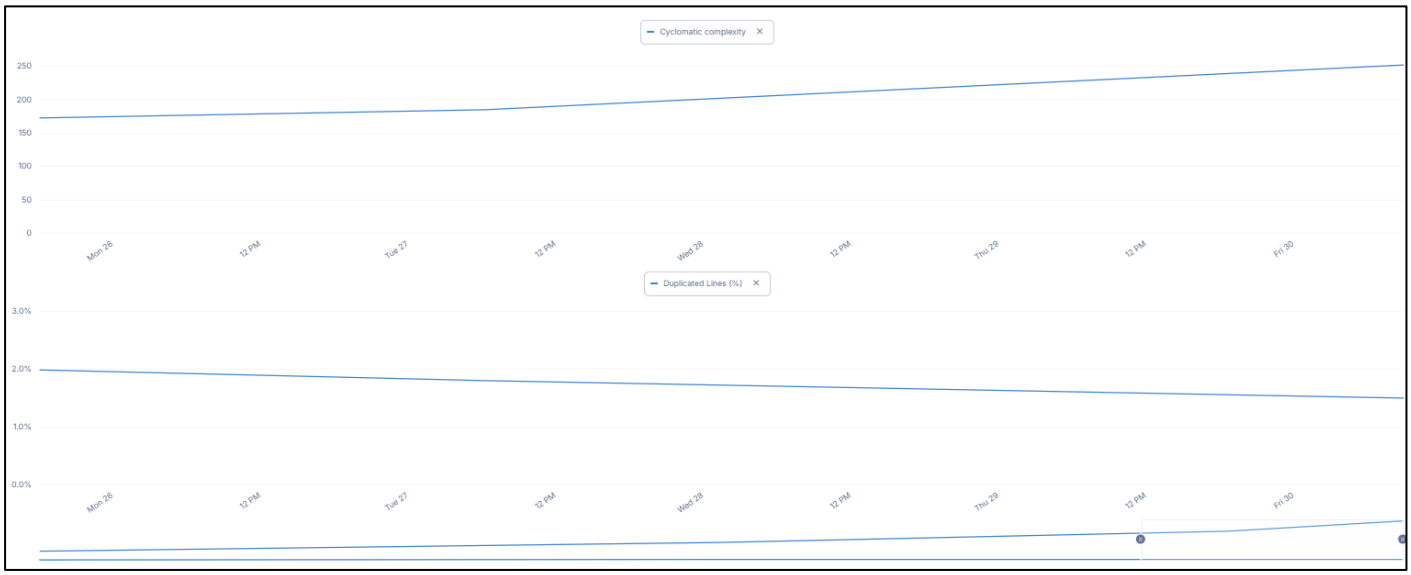


Figura 5: Porcentaje de Código Repetido y Complejidad Ciclomática

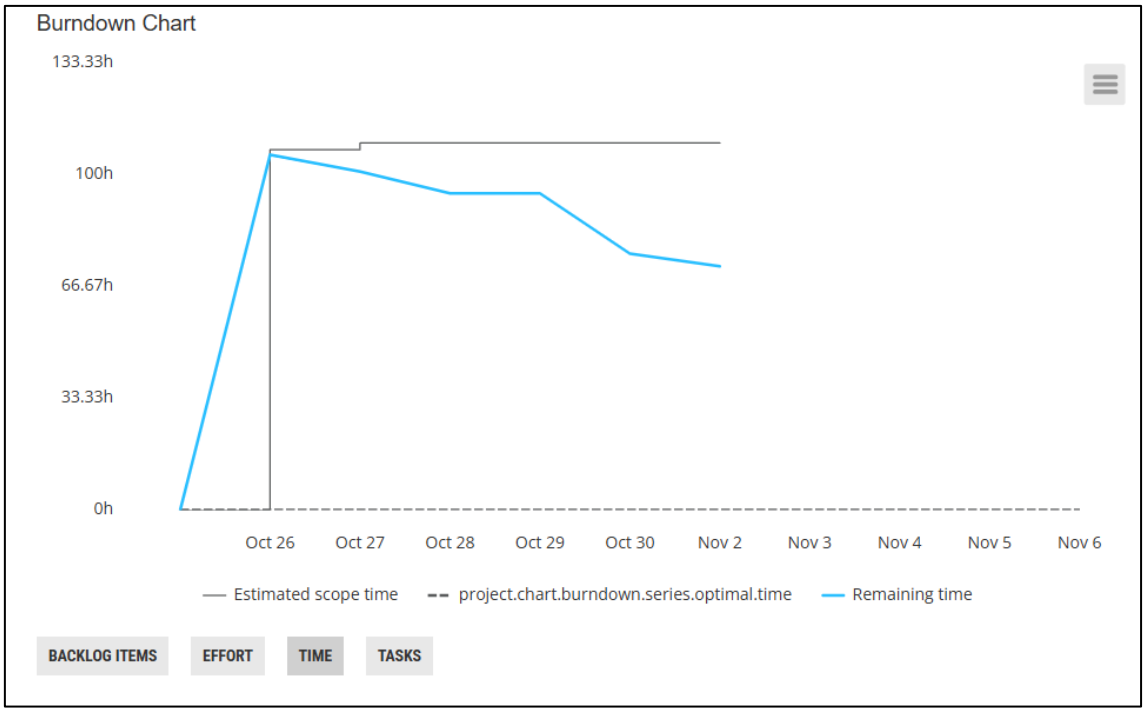


Figura 6: Burndown Chart

3. Análisis de Estado y Plan de Acción

3.1. Análisis del Estado Actual del Producto

Analizando la *Figura 1*, podemos observar que el Quality Gate pasa satisfactoriamente. Si analizamos individualmente cada uno de los tipos de Issues, encontramos únicamente 18 issues de Mantenibilidad, de los cuales 8 son de severidad media, 8 de severidad baja y 2 son meramente informativos (Ver *Figura 2*). Por otro lado, de los tipos de seguridad y fiabilidad no existe ningún issue (Ver *Figura 3*). Además, el esfuerzo de solucionar todos los problemas encontrados es bajo, necesitándose un total de 1 hora y 36 minutos para llevar a cabo el arreglo de todos ellos.

En la Figura 4, se observa la evolución de issues en donde apenas ha habido variación. Inicialmente el número issues ha aumentado a 21, mientras que el día 30 de octubre se han vuelto a ver reducidos a 18.

Para analizar cómo de mantenible es el código, se han tomado capturas de la evolución del porcentaje de código repetido y de la complejidad ciclomática. En la *Figura 5* se puede observar que la complejidad se ha visto aumentada de 112 (Valor tomado en el sprint anterior), a más del doble, 252 puntos. Este aumento de la complejidad ciclomática probablemente se deba a la introducción de nuevos métodos requeridos para completar la funcionalidad solicitada en las nuevas historias de usuario. En la misma *Figura 5*, se puede observar que el porcentaje de código duplicado ha disminuido de un 2.9% a un 1.5%.

Con este análisis, se puede concluir que la calidad del producto es buena y la deuda técnica está controlada. Aunque la complejidad ciclomática ha aumentado de forma natural debido a la adición de nuevas funcionalidades, el porcentaje de código duplicado se ha reducido y el número de issues se mantiene estable, lo que demuestra un buen mantenimiento del proyecto.

3.2. Plan de Acción

Para determinar el plan de acción, en primer lugar, se analizará si hay tiempo disponible para realizar las mejoras durante este sprint. Para ello, se toma como referencia el Burndown Chart, representado en la *Figura 6*. Si analizamos dicho gráfico, se puede observar que el ritmo de trabajo es superior al óptimo, además la resolución de los issues tiene un effort bajo, como la se ha mencionada anteriormente. Por dichas razones, se puede asumir la realización de trabajos de mejora del producto.

Para solventar todas las incidencias identificadas por Sonar, cómo ya se ha mencionado, se requieren menos de 2 horas, y viendo el ritmo de trabajo del grupo se dispone de margen suficiente para llevar a cabo la solución de todas las incidencias sin poner en riesgo la entrega del producto.

Como se pueden llevar a cabo todas las mejoras en este sprint, no es necesario darle prioridad a ninguna de ellas. En el Plan de Mejora se incorpora la corrección de las siguientes 16 de las 18 incidencias (las 2 incidencias restantes no se llevan a cabo porque son issues informativas):

- Añadir un constructor privado para evitar uno público (Severidad Media, 5 min de esfuerzo), este issue hay que corregirle en 3 ocasiones distintas:

- Añadir constructor privado de RepositoriesModule.
- Añadir constructor privado de Utils.
- Añadir constructor privado de CityBikesService.

- Eliminar bloques de líneas comentadas (Severidad Media, 5 min de esfuerzo), este issue hay que corregirle en 4 ocasiones distintas:

- En la línea 181 de la clase MainPresenter
- En la línea 203 de la clase MainPresenter
- En la línea 5 de themes.xml
- En la línea 9 de backup_rules.xml

- Eliminar una variable no usada (Severidad Media, 1 min de esfuerzo): Existe una variable context en la clase ExampleTest, que no es usada: Se podría eliminar directamente la clase dado que no tiene un papel útil para el proyecto.

- Elimina el "map" y declararlo como una variable local en los métodos correspondientes (Severidad Baja, 5 min de esfuerzo): En la clase DetailsView, existe una variable global llamada “map” que se puede cambiar a local en los métodos implicados.

- Extraer un bloque de código anidado a un método nuevo (Severidad Baja, 10 min de esfuerzo) hay que corregirle en 3 ocasiones distintas, todas ellas en la clase NetworksArrayAdapter.

- Reordenar los modificadores para cumplir con la Especificación del Lenguaje Java (Severidad Baja, 2 min de esfuerzo): En CityBikeService, existe un atributo declarado como final static …, pero el orden correcto es static final …

- Eliminar importación no utilizada 'es.unican.bicis.model.Network' (Severidad Baja, 1 min de esfuerzo): En INetworksRepository.java, el importe mencionado no es necesario.

- Invertir una condición negada e intercambiar las dos ramas del operador ternario (Severidad Baja, 2 min de esfuerzo): Dicha condición se encuentra en el método requestNerwork de la clase JsonNetworksRepository.

- Eliminar una variable local no utilizada (Severidad Baja, 5 min de esfuerzo): Esto viene dado por el mismo problema de la variable no utilizada en la clase ExampleTest.

3.3. Lista de tareas a añadir en este Sprint

Para finalizar, se añadirá en este Sprint 2 una única tarea para corregir todas las incidencias determinadas en el Plan de Acción. El motivo de que se cree una única tarea es que las correcciones a realizar son sencillas y no requieren de demasiado tiempo.

La tarea creada se llamará “Mejoras de Calidad del Código” y se estimará una duración de 2 horas, dando un pequeño margen para posibles complicaciones.